

# 1 Formalized TypeSpec and Elixir Type

|                    |          |       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Annotations</b> | <b>A</b> | $::=$ | $\overline{\text{@spec } E} \mid \varepsilon$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Expressions</b> | <b>E</b> | $::=$ | $x(\overline{T}) :: T \mid x(\overline{T}) :: T \text{ when } \overline{\alpha : T}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Types</b>       | <b>T</b> | $::=$ | $\alpha \mid T \mid T$<br>$\mid \text{any}() \mid \text{none}()$<br>$\mid \text{atom}() \mid :k$<br>$\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$<br>$\mid \text{float}() \mid \text{integer}()$<br>$\mid \text{neg\_integer}() \mid \text{non\_neg\_integer}() \mid \text{pos\_integer}()$<br>$\mid \text{tuple}() \mid \{\overline{T}\} \mid \% \{\overline{F}\}$<br>$\mid [] \mid \text{nonempty\_maybe\_improper\_list}(T, T)$<br>$\mid \llcorner \mid \llcorner :: n \llcorner \mid \llcorner :: n * n \llcorner \mid \llcorner :: n, \_ :: n * n \llcorner$<br>$\mid (\overline{T} \rightarrow T) \mid (\dots \rightarrow T)$<br>$\mid n \mid n..n'$ |
| <b>Fields</b>      | <b>F</b> | $::=$ | $\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

(where  $x$  ranges over expression variables,  $\alpha$  ranges over type variables,  $n$  ranges over integers with  $n < n'$ , and  $k$  ranges over atom names)

Figure 1: TypeSpec

|                    |          |       |                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Annotations</b> | <b>A</b> | $::=$ | $\$S \mid \$S \text{ when } \overline{\alpha : T}$                                                                                                                                                                                                                                                                                                  |
| <b>Signatures</b>  | <b>S</b> | $::=$ | $\overline{T} \rightarrow T \mid S \text{ and } S$                                                                                                                                                                                                                                                                                                  |
| <b>Types</b>       | <b>T</b> | $::=$ | $\alpha \mid n..n \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$<br>$\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$<br>$\mid \{\overline{T}\} \mid \{\overline{T}, \dots\} \mid \% \{\overline{F}\} \mid \% \{\dots, \overline{F}\}$<br>$\mid (\overline{T} \rightarrow T) \mid \text{non\_empty\_list}(T, T)$ |
| <b>Key types</b>   | <b>K</b> | $::=$ | $\text{atom}() \mid :k$<br>$\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$<br>$\mid \text{float}() \mid \text{integer}() \mid \text{binary}()$<br>$\mid \text{tuple}() \mid \text{open\_map}() \mid \text{fun}()$<br>$\mid \text{empty\_list}() \mid \text{non\_empty\_list}(\text{term}(), \text{empty\_list}())$                   |
| <b>Fields</b>      | <b>F</b> | $::=$ | $:k \Rightarrow T \mid K \Rightarrow \text{if\_set}(T)$                                                                                                                                                                                                                                                                                             |

(where  $x$  ranges over expression variables,  $\alpha$  ranges over type variables,  $n..n$  ranges over integers while invisible to the users, and  $k$  ranges over atom names)

Figure 2: Elixir Type

## 2 TypeSpec Built-in Types and Syntactic Sugars

| Built-in types                                            | Defined as                                                       |
|-----------------------------------------------------------|------------------------------------------------------------------|
| term()                                                    | any()                                                            |
| arity()                                                   | 0..255                                                           |
| *as_boolean(T)                                            | T                                                                |
| binary()                                                  | «_::_*8»                                                         |
| nonempty_binary()                                         | «_::_8, _::_*8»                                                  |
| bitstring()                                               | «_::_*1»                                                         |
| nonempty_bitstring()                                      | «_::_1, _::_*1»                                                  |
| boolean()                                                 | :true   :false                                                   |
| byte()                                                    | 0..255                                                           |
| nonempty_list(T)                                          | nonempty_maybe_improper_list(T, [])                              |
| list(T)                                                   | []   nonempty_list(T)                                            |
| nonempty_list()                                           | nonempty_list(any())                                             |
| *nonempty_improper_list(T <sub>1</sub> , T <sub>2</sub> ) | nonempty_maybe_improper_list(T <sub>1</sub> , T <sub>2</sub> )   |
| maybe_improper_list(T <sub>1</sub> , T <sub>2</sub> )     | []   nonempty_improper_list(T <sub>1</sub> , T <sub>2</sub> )    |
| maybe_improper_list()                                     | maybe_improper_list(any(), any())                                |
| nonempty_maybe_improper_list()                            | nonempty_maybe_improper_list(any(), any())                       |
| char()                                                    | 0..0x10FFFF                                                      |
| charlist()                                                | list(char())                                                     |
| nonempty_charlist()                                       | nonempty_list(char())                                            |
| fun()                                                     | (... -> any())                                                   |
| function()                                                | fun()                                                            |
| identifier()                                              | pid()   port()   reference()                                     |
| iodata()                                                  | iolist()   binary()                                              |
| iolist()                                                  | maybe_improper_list(byte()   binary()   iolist(), binary()   []) |
| keyword()                                                 | [{atom(), any()}]                                                |
| keyword(T)                                                | [{atom(), T}]                                                    |
| mfa()                                                     | {module(), atom(), arity()}                                      |
| module()                                                  | atom()                                                           |
| no_return()                                               | none()                                                           |
| node()                                                    | atom()                                                           |
| number()                                                  | integer()   float()                                              |
| struct()                                                  | %{:__struct__ => atom(), optional(atom()) => any()}              |
| timeout()                                                 | :infinity   non_neg_integer()                                    |

(built-in types with prepending asterisk (\*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

| Syntactic sugar                  | Expanded as                                 |
|----------------------------------|---------------------------------------------|
| true                             | :true                                       |
| false                            | :false                                      |
| nil                              | :nil                                        |
| [T, ...]                         | nonempty_list(T)                            |
| [T]                              | list(T)                                     |
| T <sub>1</sub> => T <sub>2</sub> | required(T <sub>1</sub> ) => T <sub>2</sub> |
| k: T                             | :k => T                                     |
| %k{F}                            | %{:__struct__ => :k, F}                     |

Table 2: TypeSpec Syntactic Sugars

### 3 Elixir Type Built-in Types and Syntactic Sugars

| Built-in types    | Defined as                         |
|-------------------|------------------------------------|
| non_empty_list(T) | non_empty_list(T, empty_list())    |
| list(T)           | empty_list() or non_empty_list(T)  |
| list()            | list(term())                       |
| improper_list()   | non_empty_list(term(), not list()) |

Table 3: Elixir Type Built-in Types

| Syntactic sugar | Expanded as       |
|-----------------|-------------------|
| {...}           | tuple()           |
| %{...}          | open_map()        |
| k: T            | :k => T           |
| K=>not_set()    | K=>if_set(none()) |

Table 4: Elixir Type Syntactic Sugars

### 4 Translation from TypeSpec to Elixir Type

$\llbracket \cdot \rrbracket$ : TypeSpec  $\rightarrow$  Elixir type

Figure 3: Translation

## 4.1 Translation in Annotations

$$\begin{aligned}
\llbracket @spec \bar{E} \rrbracket &= \begin{cases} \$ \llbracket E \rrbracket & \text{if } \llbracket @spec \bar{E} \rrbracket \approx \llbracket @spec E \rrbracket \\ \$ \llbracket \bar{E}, E' \rrbracket & \text{if } \llbracket @spec \bar{E} \rrbracket \approx \llbracket @spec \bar{E}, @spec E' \rrbracket \end{cases} \\
\llbracket E \rrbracket &= \begin{cases} \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} & \text{if } \llbracket E \rrbracket \approx \llbracket x(\bar{T}_1) :: T_2 \text{ when } \overline{\alpha : T_3} \rrbracket \\ \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket & \text{otherwise} \end{cases} \\
\llbracket \bar{E}, E' \rrbracket &= \begin{cases} \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket [\alpha' := \beta'] \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\beta' : \llbracket T'_3 \rrbracket} & \text{if } \llbracket E_i \rrbracket \approx \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ & \text{and } \llbracket E' \rrbracket \approx \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \overline{\alpha' : \llbracket T'_3 \rrbracket} \\ & \overline{\alpha} \cap \overline{\alpha'} \neq \emptyset \\ \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\alpha' : \llbracket T'_3 \rrbracket} & \text{if } \llbracket E_i \rrbracket \approx \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ & \text{and } \llbracket E' \rrbracket \approx \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \overline{\alpha' : \llbracket T'_3 \rrbracket} \\ \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases} \\
\llbracket @spec \bar{E}, @spec E' \rrbracket &= \begin{cases} \$ \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket E' \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \\ \$ \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket E' \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} & \text{if } \llbracket E_i \rrbracket \mid \llbracket E' \rrbracket \approx \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ \$ \bigcap (\llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket) \text{ and } \llbracket E' \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} & \text{if } \llbracket E_i \rrbracket \mid \llbracket E' \rrbracket \approx \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \end{cases} \\
\llbracket \epsilon \rrbracket &=
\end{aligned}$$

Figure 4: Translation in Annotations

## 4.2 Translation in Types

```

 $\llbracket \alpha \rrbracket = \alpha$ 
 $\llbracket T_1 | T_2 \rrbracket = \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket$ 
 $\llbracket \text{any}() \rrbracket = \text{term}()$ 
 $\llbracket \text{none}() \rrbracket = \text{none}()$ 
 $\llbracket \text{atom}() \rrbracket = \text{atom}()$ 
 $\llbracket :k \rrbracket = :k$ 
 $\llbracket \text{pid}() \rrbracket = \text{pid}()$ 
 $\llbracket \text{port}() \rrbracket = \text{port}()$ 
 $\llbracket \text{reference}() \rrbracket = \text{reference}()$ 
 $\llbracket \text{float}() \rrbracket = \text{float}()$ 
 $\llbracket \text{integer}() \rrbracket = \text{integer}()$ 
 $\llbracket \text{neg\_integer} \rrbracket = -\text{int} - 1$ 
 $\llbracket \text{non\_neg\_integer} \rrbracket = 0 - \text{inf}$ 
 $\llbracket \text{pos\_integer} \rrbracket = 1 - \text{inf}$ 
 $\llbracket \text{tuple}() \rrbracket = \text{tuple}()$ 
 $\llbracket \{\bar{T}\} \rrbracket = \{\llbracket \bar{T} \rrbracket\}$ 
 $\llbracket \% \{\bar{F}\} \rrbracket = \% \{\llbracket \bar{F} \rrbracket\}$ 
 $\llbracket [] \rrbracket = \text{empty\_list}()$ 
 $\llbracket \text{nonempty\_maybe\_improper\_list}(T_1, T_2) \rrbracket = \text{non\_empty\_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket)$ 
 $\llbracket \langle \rangle \rrbracket = \text{undefined}$ 
 $\llbracket \langle \_ : n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket \langle \_ : \_ * n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket \langle \_ : n, \_ : \_ * n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket (\bar{T} \rightarrow T) \rrbracket = (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket)$ 
 $\llbracket (\dots \rightarrow T) \rrbracket = \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{undefined} & \text{otherwise} \end{cases}$ 
 $\llbracket n \rrbracket = n - n$ 
 $\llbracket n..n' \rrbracket = n - n'$ 

```

Figure 5: Translation in Types

## 4.3 Translation in Fields

$$\begin{aligned}
\llbracket \text{required}(T_1) \Rightarrow T_2 \rrbracket &= \begin{cases} :k_1 \Rightarrow \llbracket T_2 \rrbracket, \dots, :k_n \Rightarrow \llbracket T_2 \rrbracket & \text{if } \llbracket T_1 \rrbracket \approx :k \\ K_1 \Rightarrow \text{if\_set}(\llbracket T_2 \rrbracket), \dots, K_n \Rightarrow \text{if\_set}(\llbracket T_2 \rrbracket) & \text{if } \llbracket T_1 \rrbracket \approx K_i \text{ where } K_i \neq :k \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket \text{optional}(T_1) \Rightarrow T_2 \rrbracket &= \begin{cases} K_1 \Rightarrow \text{if\_set}(\llbracket T_2 \rrbracket), \dots, K_n \Rightarrow \text{if\_set}(\llbracket T_2 \rrbracket) & \text{if } \llbracket T_2 \rrbracket \approx K_1 \text{ or } \dots \text{ or } K_n \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 6: Translation in Fields